

データベース設計書

ER図

コア機能（ユーザー・プロフィール・認証）

```
erDiagram
    users ||--ol profiles : has
    users ||--ol student_verifications : has
    users ||--o{ social_connections : has
    users ||--ol property_preferences : has
    users ||--ol lifestyles : has
    users ||--o{ user_interests : has
    interest_tags ||--o{ user_interests : referenced_by
    users ||--o{ lifestyle_assessments : has

    users {
        uuid id PK
        string email UK
        string password_hash
        string phone_number UK
        boolean phone_verified
        boolean email_verified
        boolean is_university_email
        integer trust_score
        string status
        datetime last_login_at
        datetime inserted_at
        datetime updated_at
    }

    profiles {
        uuid id PK
        uuid user_id FK
        string nickname
        integer age
        string gender
        string user_type
        string university_name
        string faculty
        string department
        integer grade
        string campus
        string occupation
        text bio
        string video_url
        string avatar_url
        datetime inserted_at
        datetime updated_at
    }
```

```
student_verifications {
  uuid id PK
  uuid user_id FK
  string student_card_image_url
  string university_name
  string status
  datetime verified_at
  datetime inserted_at
}

social_connections {
  uuid id PK
  uuid user_id FK
  string provider
  string provider_user_id
  datetime inserted_at
}

property_preferences {
  uuid id PK
  uuid user_id FK
  decimal min_rent
  decimal max_rent
  decimal max_initial_cost
  array preferred_prefectures
  array preferred_cities
  array preferred_stations
  integer max_commute_time
  array room_types
  integer min_building_age
  integer max_building_age
  boolean requires_auto_lock
  boolean requires_separate_bath_toilet
  boolean requires_south_facing
  text other_requirements
  string move_in_timing
  datetime inserted_at
  datetime updated_at
}

lifestyles {
  uuid id PK
  uuid user_id FK
  string sleep_schedule
  integer cleanliness_level
  boolean smoking
  boolean e_cigarette
  string pets
  string guest_frequency
  integer noise_tolerance
  integer socializing_preference
}
```

```

    string home_time
    string cooking_frequency
    datetime inserted_at
    datetime updated_at
}

interest_tags {
    uuid id PK
    string name UK
    string category
    datetime inserted_at
}

user_interests {
    uuid user_id FK
    uuid interest_tag_id FK
    datetime inserted_at
}

lifestyle_assessments {
    uuid id PK
    uuid user_id FK
    integer question_id
    integer answer_value
    datetime completed_at
    datetime inserted_at
}

```

マッチング機能

```

erDiagram
    users ||--o{ match_requests_sender : sends
    users ||--o{ match_requests_receiver : receives
    match_requests ||--o{ matched_pairs : creates
    matched_pairs ||--o{ messages : has
    users ||--o{ messages_sender : sends
    users ||--o{ messages_receiver : receives
    matched_pairs ||--o{ property_candidates : has
    property_candidates ||--o{ viewing_schedules : has
    matched_pairs ||--o{ pair_preferences : has

    match_requests {
        uuid id PK
        uuid sender_user_id FK
        uuid receiver_user_id FK
        string status
        text message
        integer compatibility_score
        datetime expires_at
        datetime inserted_at
        datetime updated_at
    }

```

```
}

matched_pairs {
  uuid id PK
  uuid user1_id FK
  uuid user2_id FK
  uuid match_request_id FK
  string status
  datetime matched_at
  datetime dissolved_at
  datetime inserted_at
}

messages {
  uuid id PK
  uuid sender_user_id FK
  uuid receiver_user_id FK
  uuid matched_pair_id FK
  text content
  boolean is_flagged
  datetime read_at
  datetime inserted_at
}

property_candidates {
  uuid id PK
  uuid matched_pair_id FK
  string source_url
  string title
  text address
  decimal rent
  decimal initial_cost
  string room_type
  integer building_age
  string station_name
  integer station_walk_minutes
  array image_urls
  text description
  integer user1_rating
  integer user2_rating
  text user1_comment
  text user2_comment
  string status
  datetime inserted_at
  datetime updated_at
}

pair_preferences {
  uuid id PK
  uuid matched_pair_id FK
  decimal agreed_min_rent
  decimal agreed_max_rent
```

```

        decimal agreed_max_initial_cost
        array agreed_areas
        array agreed_stations
        array agreed_room_types
        array priority_order
        datetime inserted_at
        datetime updated_at
    }

    viewing_schedules {
        uuid id PK
        uuid property_candidate_id FK
        datetime scheduled_date
        text location
        text notes
        string status
        datetime inserted_at
    }

```

コミュニティ・レビュー機能

erDiagram

```

users ||--o{ reviews_reviewer : writes
users ||--o{ reviews_reviewee : receives
matched_pairs ||--o{ reviews : about
users ||--o{ reports_reporter : reports
users ||--o{ reports_reported : reported_by
users ||--o{ blocked_users_blocker : blocks
users ||--o{ blocked_users_blocked : blocked_by
users ||--o{ favorite_users_user : favorites
users ||--o{ favorite_users_favorite : favorited_by
users ||--o{ view_history_viewer : views
users ||--o{ view_history_viewed : viewed_by
users ||--o{ notifications : receives
users ||--o{ university_posts : writes
university_posts ||--o{ university_post_comments : has

```

```

reviews {
    uuid id PK
    uuid reviewer_user_id FK
    uuid reviewee_user_id FK
    uuid matched_pair_id FK
    integer cleanliness_rating
    integer communication_rating
    integer rule_compliance_rating
    integer financial_reliability_rating
    integer overall_rating
    text positive_comment
    text improvement_comment
    text advice_for_next
    string visibility
}

```

```
        datetime inserted_at
    }

    reports {
        uuid id PK
        uuid reporter_user_id FK
        uuid reported_user_id FK
        string reason
        text description
        string status
        datetime resolved_at
        datetime inserted_at
    }

    blocked_users {
        uuid blocker_user_id FK
        uuid blocked_user_id FK
        datetime blocked_at
        datetime inserted_at
    }

    favorite_users {
        uuid user_id FK
        uuid favorite_user_id FK
        datetime inserted_at
    }

    view_history {
        uuid id PK
        uuid user_id FK
        uuid viewed_user_id FK
        datetime viewed_at
        datetime inserted_at
    }

    notifications {
        uuid id PK
        uuid user_id FK
        string type
        string title
        text content
        string link_url
        boolean is_read
        datetime read_at
        datetime inserted_at
    }

    university_posts {
        uuid id PK
        uuid user_id FK
        string university_name
        string title
    }
}
```

```
text content
string category
datetime inserted_at
datetime updated_at
}

university_post_comments {
  uuid id PK
  uuid post_id FK
  uuid user_id FK
  text content
  datetime inserted_at
}
```

テーブル一覧

1. ユーザー管理（4テーブル）

- **users:** ユーザー基本情報・認証
- **profiles:** プロフィール詳細
- **student_verifications:** 学生証認証
- **social_connections:** SNS連携

2. 希望条件・ライフスタイル（5テーブル）

- **property_preferences:** 物件希望条件
- **lifestyles:** ライフスタイル情報
- **interest_tags:** 興味・趣味タグ
- **user_interests:** ユーザーと興味タグの関連
- **lifestyle_assessments:** ライフスタイル診断結果

3. マッチング機能（6テーブル）

- **match_requests:** マッチングリクエスト
- **matched_pairs:** マッチング成立ペア
- **messages:** メッセージ
- **property_candidates:** 物件候補リスト
- **pair_preferences:** ペアの共通希望条件
- **viewing_schedules:** 内見スケジュール

4. コミュニティ機能（7テーブル）

- **reviews:** レビュー・評価
- **reports:** 通報
- **blocked_users:** ブロックリスト
- **favorite_users:** お気に入りユーザー
- **view_history:** 閲覧履歴
- **notifications:** 通知
- **university_posts:** 大学コミュニティ投稿
- **university_post_comments:** 投稿へのコメント

合計: 22テーブル

主要な設計方針

1. ルームメイトファースト

- 物件テーブルは存在せず、**ユーザー間のマッチングが中心**
- 物件は `property_candidates`（候補）として、マッチング後に2人で共有

2. セキュリティ

- **UUID**をプライマリーキーとして使用（推測困難）
- パスワードは**ハッシュ化**して保存
- 信頼スコアシステムで安全性を担保

3. データ整合性

- **外部キー制約**で関連データの整合性を保証
- **Check制約**で評価値の範囲を検証（1-5、0-100等）
- **Unique制約**で重複を防止

4. パフォーマンス

- 頻繁に検索されるカラムに**インデックス**を設定
- PostgreSQLの**配列型**を活用（`preferred_cities`等）

5. 拡張性

- **enum型ではなくstring型**を使用（柔軟性重視）
- タイムスタンプで履歴管理
- ソフトデリート可能な設計

次のステップ

1.  マイグレーション作成完了
2.  Ectoスキーマモジュールの作成
3.  Changesetによるバリデーション実装
4.  Context層のビジネスロジック実装
5.  API実装